Danmarks
Tekniske
Universitet

Lecture 1

Microcontroller Development for Advanced IoT using Embedded C (34373)

Author

Hákon Hlynsson - s233240

February 8, 2026

Contents

1 Execise **1**

References **6**

2 Appendix **7**

1 Execise

EX1: Look up the HAL_Delay() function in the HAL manual and explain what it does ?

According to the HAL manual (page 32), the HAL_Delay(Delay) function is used to pause code execution for a specific duration, defined in milliseconds by the input parameter "Delay".

The function operates by relying on the SysTick ISR, which triggers every 1ms to increment a global tick variable. HAL_Delay() then continuously polls this variable,and thereby stopping the execution of code until the variable has increased the same amount as the specified "Delay".

EX2: Use the defined names found in gpio.c instead, and change frequency to 100ms.

Listing 1: LED toggles at 100ms

```c
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();

  while (1)
  {
    HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin); // Toggle LED
    HAL_Delay(100);                             // Delay 100ms
  }
}
```

Listing 2: Initializing the LED

```c
void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_RESET);
  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = LD2_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
```

```
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(LD2_GPIO_Port, &GPIO_InitStruct);
}
```

EX3: Look in HAL manual for another way to manipulate the GPIO (set/reset) and use this to make LED blink.

Listing 3: LED blink useing (set/reset)

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();

  while (1)
  {
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin,GPIO_PIN_RESET);
  HAL_Delay(100);
  HAL_GPIO_WritePin(LD2_GPIO_Port, LD2_Pin, GPIO_PIN_SET);
  HAL_Delay(100);
  }
}
```

EX4: Create a variable X (char, int or uint8_t type) that counts up from 0. With each step the LED should blink X times (speed 100ms). Incorporate a pause of 1 sec between each increase of X. 0, 1, 2, 3, 4 etc blinks..

Listing 4: EX4

```c
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();

  uint8_t X = 0;

  while (1)
  {
  for (int i = 0; i < X; i++){
      //breakpoint is place here fore next exercise
      HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_SET);
      HAL_Delay(100);
      HAL_GPIO_WritePin(LD2_GPIO_Port,LD2_Pin,GPIO_PIN_RESET);
      HAL_Delay(100);
      }
  HAL_Delay(1000);
  X++;
  }
}
```

EX5: Use the Debug function to monitor and observe value of X and steps through code. Make screenshots.

To see where the breakpoint is placed, see the code in exercise 4 above.

Expression	Type	Value
X	uint8_t	1 '\001'
i	int	0
Add new expression		

Figure 1: First time at the breakpoint

Expression	Type	Value
X	uint8_t	2 '\002'
i	int	0

Figure 2: Second time at the breakpoint

Expression	Type	Value
X	uint8_t	2 '\002'
i	int	1
Add new expression		

Figure 3: Third time at the breakpoint

Expression	Type	Value
X	uint8_t	3 '\003'
i	int	0
Add new expression		

Figure 4: Fourth time at the breakpoint

Expression	Type	Value
X	uint8_t	3 '\003'
i	int	1
Add new expression		

Figure 5: Fifth time at the breakpoint

Expression	Type	Value
X	uint8_t	3 '\003'
i	int	2
Add new expression		

Figure 6: Sixth time at the breakpoint

EX6: Use CubeMX to configure one more GPIO to make an externally added LED blink. Upload code, screenshots from CubeMX and images of LED

I have now changed the code so that the LED is external instead of the internal LED. The chosen pin is PC9. The code can be seen below.

Listing 5: Interlization of LED

```
    void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOF_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();

```

```
  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_9, GPIO_PIN_RESET);

  /*Configure GPIO pin : LD2_Pin */
  GPIO_InitStruct.Pin = GPIO_PIN_9;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
}
```

Listing 6: Execise 4 running on an external LED

```
int main(void)
{
  HAL_Init();
  SystemClock_Config();
  MX_GPIO_Init();


  uint8_t X = 0;
  while (1)
  {
    for (int i = 0; i < X; i++){
         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_9, GPIO_PIN_SET);
         HAL_Delay(100);
         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_9, GPIO_PIN_RESET);
         HAL_Delay(100);
    }
    HAL_Delay(1000);
    X++;
  }
}
```

Link to GitHub, where the report and video are stored in "Lecture 1" and the code in "Hakon_projekt":
https://github.com/Hakon2003DTU/Hakon_master_notes/tree/main/1_Semester_(Spring_2026)/34373_Microcontroler_development_for_advanced_IoT_using_embedded_C

References

2 Appendix